
Program Semantics

Lecture 2 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

Lecture 1: what static analysis is, why it matters, some industrial impact (Infer).

Today: the *meaning* of a program — its *semantics*.

Why does a static analyzer need semantics at all?

- An analyzer computes an *approximation* of program behaviour.
- *Approximation of what?* We need a precise, mathematical reference: the semantics.
- Soundness = “the analyzer never contradicts the semantics”.

Why a semantics is needed

Correctness can only be stated with respect to a definition of what programs do.

Roadmap

The IMP language

Standard semantics

Rice's theorem

Collecting semantics

§ 1

The IMP language

A minimal imperative language: IMP

We fix one small language for the whole course.

Arithmetic expressions $a \in \text{AExp}$

$$a ::= n \mid x \mid a_1 + a_2 \mid a_1 - a_2 \mid a_1 \times a_2$$

Boolean expressions $b \in \text{BExp}$

$$b ::= \text{true} \mid \text{false} \mid a_1 \bowtie a_2 \mid \neg b \mid b_1 \wedge b_2 \mid b_1 \vee b_2 \quad \bowtie \in \{\leq, \geq, <, >, =, \neq\}$$

Commands $c \in \text{Cmd}$

$$c ::= \text{skip} \mid x := a \mid c_1 ; c_2 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \mid \text{while } b \text{ do } c$$

where $n \in \mathbb{Z}$, $x \in \text{Var}$ (a finite set of variables).

Definition 1 (State)

A *state* (or store) is a total map from variables to values:

$$\sigma \in \Sigma \triangleq \text{Var} \rightarrow \mathbb{Z}.$$

Notation:

- $\sigma(x)$ — value of x in state σ ;
- $\sigma[x \mapsto v]$ — update: same as σ except x now maps to v :

$$\sigma[x \mapsto v](y) = \begin{cases} v & \text{if } y = x, \\ \sigma(y) & \text{otherwise.} \end{cases}$$

For now: no runtime errors, and unbounded integers.

Semantics of expressions (no side effects)

Expressions are pure: their value depends only on the current state.

Arithmetic: $\llbracket a \rrbracket : \Sigma \rightarrow \mathbb{Z}$

$$\llbracket n \rrbracket \sigma = n$$

$$\llbracket a_1 + a_2 \rrbracket \sigma = \llbracket a_1 \rrbracket \sigma + \llbracket a_2 \rrbracket \sigma$$

$$\llbracket a_1 \times a_2 \rrbracket \sigma = \llbracket a_1 \rrbracket \sigma \times \llbracket a_2 \rrbracket \sigma$$

$$\llbracket x \rrbracket \sigma = \sigma(x)$$

$$\llbracket a_1 - a_2 \rrbracket \sigma = \llbracket a_1 \rrbracket \sigma - \llbracket a_2 \rrbracket \sigma$$

Boolean: $\llbracket b \rrbracket : \Sigma \rightarrow \mathbb{B}$

$$\llbracket \text{true} \rrbracket \sigma = \text{tt}$$

$$\llbracket a_1 \bowtie a_2 \rrbracket \sigma = (\llbracket a_1 \rrbracket \sigma \bowtie \llbracket a_2 \rrbracket \sigma)$$

$$\llbracket b_1 \wedge b_2 \rrbracket \sigma = \llbracket b_1 \rrbracket \sigma \wedge \llbracket b_2 \rrbracket \sigma$$

$$\llbracket \text{false} \rrbracket \sigma = \text{ff}$$

$$\llbracket \neg b \rrbracket \sigma = \neg \llbracket b \rrbracket \sigma$$

$$\llbracket b_1 \vee b_2 \rrbracket \sigma = \llbracket b_1 \rrbracket \sigma \vee \llbracket b_2 \rrbracket \sigma$$

§2

Standard semantics

Three ways to give meaning to commands

Three standard styles:

Operational

How it runs, step by step.

A transition system on configurations.

Denotational

What it computes.

A (partial) function on states.

Axiomatic

What it guarantees.

Pre/post-condition assertions.

We will present these three forms very briefly, then introduce a *fourth* form better suited to abstract interpretation: the [collecting semantics](#).

Big-step operational semantics

$\langle c, \sigma \rangle \Downarrow \sigma'$ reads “ c started in σ terminates in σ' ”.

$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \Downarrow \sigma} \text{ SKIP} \qquad \frac{}{\langle x := a, \sigma \rangle \Downarrow \sigma[x \mapsto \llbracket a \rrbracket \sigma]} \text{ ASSIGN} \\[10pt] \frac{\langle c_1, \sigma \rangle \Downarrow \sigma' \quad \langle c_2, \sigma' \rangle \Downarrow \sigma''}{\langle c_1 ; c_2, \sigma \rangle \Downarrow \sigma''} \text{ SEQ} \\[10pt] \frac{\llbracket b \rrbracket \sigma = \text{tt} \quad \langle c_1, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} \text{ IF-TRUE} \qquad \frac{\llbracket b \rrbracket \sigma = \text{ff} \quad \langle c_2, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} \text{ IF-FALSE} \\[10pt] \frac{\llbracket b \rrbracket \sigma = \text{tt} \quad \langle c, \sigma \rangle \Downarrow \sigma' \quad \langle \text{while } b \text{ do } c, \sigma' \rangle \Downarrow \sigma''}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma''} \text{ WHILE-TRUE} \\[10pt] \frac{\llbracket b \rrbracket \sigma = \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma} \text{ WHILE-FALSE} \end{array}$$

Example: a big-step derivation

A big-step judgement is justified by a **derivation tree**, not by a linear sequence of steps. Let $w \equiv \text{while } x \leq 2 \text{ do } x := x+1$. From $[x \mapsto 2]$:

$$\frac{\frac{\frac{\llbracket x \leq 2 \rrbracket [x \mapsto 2] = \text{tt}}{\langle x := x+1, [x \mapsto 2] \rangle \Downarrow [x \mapsto 3]} \text{ ASSIGN} \quad \frac{\llbracket x \leq 2 \rrbracket [x \mapsto 3] = \text{ff}}{\langle w, [x \mapsto 3] \rangle \Downarrow [x \mapsto 3]} \text{ W-FALSE}}{\langle w, [x \mapsto 2] \rangle \Downarrow [x \mapsto 3]} \text{ W-TRUE}$$

Starting from $[x \mapsto 0]$ instead would nest W-TRUE three times — the tree grows with the number of iterations.

Small-step operational semantics

Big-step hides what happens *during* a run; small-step exposes it one step at a time. A *configuration* $\langle c, \sigma \rangle$ pairs the code left to run with the current state; a step yields another configuration, or a *terminal state*.

$$\frac{}{\langle \text{skip}, \sigma \rangle \rightsquigarrow \sigma} \text{ SKIP}$$

$$\frac{}{\langle x := a, \sigma \rangle \rightsquigarrow \sigma[x \mapsto \llbracket a \rrbracket \sigma]} \text{ ASSIGN}$$

$$\frac{\langle c_1, \sigma \rangle \rightsquigarrow \langle c'_1, \sigma' \rangle}{\langle c_1 ; c_2, \sigma \rangle \rightsquigarrow \langle c'_1 ; c_2, \sigma' \rangle} \text{ SEQ}$$

$$\frac{\langle c_1, \sigma \rangle \rightsquigarrow \sigma'}{\langle c_1 ; c_2, \sigma \rangle \rightsquigarrow \langle c_2, \sigma' \rangle} \text{ SEQ-FIN}$$

$$\frac{\llbracket b \rrbracket \sigma = \text{tt}}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \rightsquigarrow \langle c_1, \sigma \rangle} \text{ IF-TRUE}$$

$$\frac{\llbracket b \rrbracket \sigma = \text{ff}}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \rightsquigarrow \langle c_2, \sigma \rangle} \text{ IF-FALSE}$$

$$\frac{\llbracket b \rrbracket \sigma = \text{tt}}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightsquigarrow \langle c ; \text{while } b \text{ do } c, \sigma \rangle} \text{ WHILE-TRUE}$$

$$\frac{\llbracket b \rrbracket \sigma = \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightsquigarrow \sigma} \text{ WHILE-FALSE}$$

Example: a small-step derivation

Let

$$w \triangleq \text{while } x < 2 \text{ do } x := x + 1, \quad \sigma_i \triangleq [x \mapsto i].$$

Starting from σ_0 , the complete derivation is the reduction chain

$$\langle w, \sigma_0 \rangle \xrightarrow{\text{WHILE-TRUE}} \langle x := x + 1; w, \sigma_0 \rangle$$

because $\llbracket x < 2 \rrbracket \sigma_0 = \text{tt}$

$$\xrightarrow{\text{SEQ-FIN}} \langle w, \sigma_1 \rangle$$

because $\langle x := x + 1, \sigma_0 \rangle \rightsquigarrow \sigma_1$

$$\xrightarrow{\text{WHILE-TRUE}} \langle x := x + 1; w, \sigma_1 \rangle$$

because $\llbracket x < 2 \rrbracket \sigma_1 = \text{tt}$

$$\xrightarrow{\text{SEQ-FIN}} \langle w, \sigma_2 \rangle$$

because $\langle x := x + 1, \sigma_1 \rangle \rightsquigarrow \sigma_2$

$$\xrightarrow{\text{WHILE-FALSE}} \sigma_2$$

because $\llbracket x < 2 \rrbracket \sigma_2 = \text{ff}$.

Small-step with an explicit continuation

The previous semantics stores “what remains to be done” in the residual command. An abstract machine can store it explicitly in a *continuation*:

$$\kappa ::= \text{Halt} \mid \text{KSeq}(c, \kappa) \mid \text{KLoop}(b, c, \kappa).$$

Its control state distinguishes two situations:

$$\text{Enter}(c, \sigma, \kappa) \quad | \quad \text{Done}(\sigma, \kappa).$$

- Enter means that c still has to be executed;
- Done passes the resulting state to the continuation.

For sequencing:

$$\begin{aligned} \text{Enter}(c_1; c_2, \sigma, \kappa) &\longrightarrow_K \text{Enter}(c_1, \sigma, \text{KSeq}(c_2, \kappa)), \\ \text{Done}(\sigma', \text{KSeq}(c_2, \kappa)) &\longrightarrow_K \text{Enter}(c_2, \sigma', \kappa). \end{aligned}$$

Continuation-machine transitions

Commands that finish immediately produce a Done state:

$$\begin{aligned}\text{Enter}(\text{skip}, \sigma, \kappa) &\longrightarrow_K \text{Done}(\sigma, \kappa), \\ \text{Enter}(x := a, \sigma, \kappa) &\longrightarrow_K \text{Done}(\sigma[x \mapsto \llbracket a \rrbracket \sigma], \kappa).\end{aligned}$$

A conditional immediately enters the selected branch:

$$\text{Enter}(\text{if } b \text{ then } c_1 \text{ else } c_2, \sigma, \kappa) \longrightarrow_K \begin{cases} \text{Enter}(c_1, \sigma, \kappa) & \text{if } \llbracket b \rrbracket \sigma = \text{tt}, \\ \text{Enter}(c_2, \sigma, \kappa) & \text{if } \llbracket b \rrbracket \sigma = \text{ff}. \end{cases}$$

A loop frame remembers that the guard must be tested again after its body:

$$\begin{aligned}\text{Enter}(\text{while } b \text{ do } c, \sigma, \kappa) &\longrightarrow_K \text{Enter}(c, \sigma, \text{KLoop}(b, c, \kappa)) && \text{if } \llbracket b \rrbracket \sigma = \text{tt}, \\ \text{Enter}(\text{while } b \text{ do } c, \sigma, \kappa) &\longrightarrow_K \text{Done}(\sigma, \kappa) && \text{if } \llbracket b \rrbracket \sigma = \text{ff}, \\ \text{Done}(\sigma', \text{KLoop}(b, c, \kappa)) &\longrightarrow_K \text{Enter}(\text{while } b \text{ do } c, \sigma', \kappa).\end{aligned}$$

The terminal configuration is $\text{Done}(\sigma, \text{Halt})$.

We will use a similar machine structure in the project.

The same example with a continuation

Let

$$w \triangleq \text{while } \underbrace{x < 2}_b \text{ do } \underbrace{x := x + 1}_c, \quad \kappa \triangleq \text{KLoop}(b, c, \text{Halt}).$$

$\text{Enter}(w, \sigma_0, \text{Halt}) \longrightarrow_K \text{Enter}(c, \sigma_0, \kappa)$
 $\longrightarrow_K \text{Done}(\sigma_1, \kappa)$
 $\longrightarrow_K \text{Enter}(w, \sigma_1, \text{Halt})$
 $\longrightarrow_K \text{Enter}(c, \sigma_1, \kappa)$
 $\longrightarrow_K \text{Done}(\sigma_2, \kappa)$
 $\longrightarrow_K \text{Enter}(w, \sigma_2, \text{Halt})$
 $\longrightarrow_K \text{Done}(\sigma_2, \text{Halt})$

because $\llbracket b \rrbracket \sigma_0 = \text{tt}$
because c assigns $x + 1$ to x
because κ resumes the loop
because $\llbracket b \rrbracket \sigma_1 = \text{tt}$
because c assigns $x + 1$ to x
because κ resumes the loop
because $\llbracket b \rrbracket \sigma_2 = \text{ff}$.

Terminating, diverging, or stuck

A maximal run from $\langle c, \sigma \rangle$ has exactly **three** fates:

- **terminates**: $\langle c, \sigma \rangle \rightsquigarrow^* \sigma'$ — reaches a terminal state;
- **diverges**: an infinite reduction chain;
- **stuck**: reaches a $\langle c', \sigma' \rangle$ that is *not* terminal yet has *no successor* (no rule applies) — **something unexpected happened**.

In *pure* IMP, stuck never happens

Expression evaluation $\llbracket a \rrbracket, \llbracket b \rrbracket$ is **total**, so every non-terminal configuration reduces. The run is a clean dichotomy terminate / diverge — which is why the equivalences hold.

Stuck = runtime error, once evaluation is partial

Add division with $\llbracket a_1 / a_2 \rrbracket \sigma$ *undefined* when $\llbracket a_2 \rrbracket \sigma = 0$. Then $\langle x := 1/y, [y \mapsto 0] \rangle$ is **stuck**. Later: null dereference, use-after-free (Lecture 9).

Proving a program never gets stuck = proving safety — exactly the analyzer's job.

Exercise — small-step vs. big-step

State (*without proof*) the theorem relating big-step, small-step and the continuation machine.
What changes if assignment can get stuck?

Exercise — small-step vs. big-step

State (*without proof*) the theorem relating big-step, small-step and the continuation machine.
What changes if assignment can get stuck?

Solution

Equivalence on terminating runs. For all c, σ, σ' :

$$\begin{aligned} \langle c, \sigma \rangle \Downarrow \sigma' &\iff \langle c, \sigma \rangle \rightsquigarrow^* \sigma' \\ &\iff \text{Enter}(c, \sigma, \text{Halt}) \longrightarrow_K^* \text{Done}(\sigma', \text{Halt}). \end{aligned}$$

Exercise — small-step vs. big-step

State (*without proof*) the theorem relating big-step, small-step and the continuation machine.
What changes if assignment can get stuck?

Solution

Equivalence on terminating runs. For all c, σ, σ' :

$$\begin{aligned}\langle c, \sigma \rangle \Downarrow \sigma' &\iff \langle c, \sigma \rangle \rightsquigarrow^* \sigma' \\ &\iff \text{Enter}(c, \sigma, \text{Halt}) \longrightarrow_K^* \text{Done}(\sigma', \text{Halt}).\end{aligned}$$

If assignment can get stuck, small-step distinguishes:

1. **an error will occur** when it reaches a non-terminal configuration with no successor: $\langle c, \sigma \rangle \rightsquigarrow^* \langle c', \sigma' \rangle$ and $\nexists q, \langle c', \sigma' \rangle \rightsquigarrow q$;
2. **execution does not terminate** when it produces an infinite reduction chain $\langle c, \sigma \rangle = q_0 \rightsquigarrow q_1 \rightsquigarrow q_2 \rightsquigarrow \dots$.

Big-step represents both cases in the same way: $\nexists \sigma'. \langle c, \sigma \rangle \Downarrow \sigma'$.

Denotational semantics — fast overview

A command *denotes* a **partial function** on states, $\llbracket c \rrbracket_D : \Sigma \rightharpoonup \Sigma$, undefined exactly where the program diverges. Compositional by construction:

$$\llbracket \text{skip} \rrbracket_D = \text{id}, \quad \llbracket x := a \rrbracket_D \sigma = \sigma[x \mapsto \llbracket a \rrbracket \sigma], \quad \llbracket c_1 ; c_2 \rrbracket_D = \llbracket c_2 \rrbracket_D \circ \llbracket c_1 \rrbracket_D,$$

and $\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket_D$ picks a branch according to $\llbracket b \rrbracket \sigma$.

The loop is the problem. Its semantics satisfies the recursive equation

$$\llbracket \text{while } b \text{ do } c \rrbracket_D \sigma = \begin{cases} \llbracket \text{while } b \text{ do } c \rrbracket_D (\llbracket c \rrbracket_D \sigma) & \text{if } \llbracket b \rrbracket \sigma = \text{tt}, \\ \sigma & \text{if } \llbracket b \rrbracket \sigma = \text{ff}. \end{cases}$$

It is defined as the **least fixpoint** of the corresponding functional:

$$\llbracket \text{while } b \text{ do } c \rrbracket_D \triangleq \text{lfp}(\lambda f. \lambda \sigma. \text{if } \llbracket b \rrbracket \sigma \text{ then } f(\llbracket c \rrbracket_D \sigma) \text{ else } \sigma).$$

A similar construction will be needed for the collecting semantics.

Axiomatic semantics: Hoare logic

Hoare triples $\{P\} c \{Q\}$: if P holds before c , and c terminates, then Q holds afterwards (*partial correctness*). P, Q are formulas over the program variables, each denoting an element of $\mathcal{P}(\Sigma)$.

The one interesting rule is the loop, and it runs on an *invariant* I — together with the *rule of consequence*:

$$\frac{\{I \wedge b\} c \{I\}}{\{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \qquad \frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}}$$

What to retain

The invariant I has to be *guessed*, which is what makes program proof costly. *Abstract interpretation computes it automatically*: the abstract fixpoint is an inductive invariant.

§3

Rice's theorem

Why approximation is unavoidable

We would love an exact analyzer: given c , answer *yes or no*: “does c ever divide by zero? / dereference null? / ...”

Theorem 2 (Rice, 1953)

Let $\mathcal{P}$ be any non-trivial property of the partial function computed by a program (i.e. true for some programs, false for others, and depending only on the input/output behaviour). Then $\mathcal{P}$ is [undecidable](#).

In particular, all undecidable:

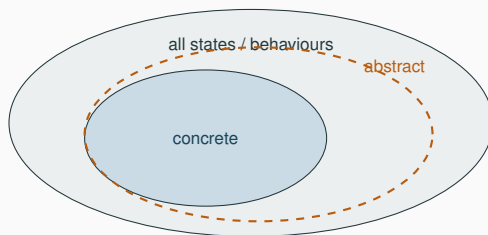
- “does this program halt on all inputs?”
- “is variable x always ≥ 0 at line 42?”
- “is this array access always in bounds?”

Rice's theorem: consequence for us

The fundamental trade-off

No algorithm can be simultaneously **sound**, **complete**, **terminating** and **automatic** on all programs.

Static analysis keeps *sound + terminating + automatic*, and *drops completeness*: it *over-approximates*.



Sound but incomplete: the analyzer may report *false alarms*, but never misses a real one.

§ 4

Collecting semantics

From “one run” to “all runs”

Operational/denotational semantics describe *a single execution* from *one* initial state.

A static analyzer must reason about **all executions** at once.

Idea

Lift the semantics to *sets of states*. At each program point, collect *the set of all states that can reach it*.

This is the *collecting semantics* — the object the abstract interpreter will approximate.

Calculational design

We formulate this semantics so that its structure is close to that of the final analyzer. The analyzer will then be derived by replacing exact operations with sound abstract ones.

Collecting semantics, functionally

Work in the powerset lattice $(\mathcal{P}(\Sigma), \subseteq)$. (Lattices will be defined in the next lecture.)

Lift each command to a function on *sets* of states, $\llbracket c \rrbracket : \mathcal{P}(\Sigma) \rightarrow \mathcal{P}(\Sigma)$:

$$\begin{aligned}\llbracket \text{skip} \rrbracket S &\triangleq S \\ \llbracket x := a \rrbracket S &\triangleq \{ \sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in S \} \\ \llbracket c_1 ; c_2 \rrbracket S &\triangleq \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket S) \\ \llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket S &\triangleq \llbracket c_1 \rrbracket (\llbracket b \rrbracket S) \cup \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket S)\end{aligned}$$

A Boolean expression induces a filter on sets of states:

$$\llbracket b \rrbracket S \triangleq \{ \sigma \in S \mid \llbracket b \rrbracket \sigma = \text{tt} \}.$$

We will also write the filter as $S \cap \{b\}$, with $\{b\} \triangleq \{ \sigma \mid \llbracket b \rrbracket \sigma = \text{tt} \}$.

The loop, again a least fixpoint

Let one loop iteration be

$$T \triangleq \llbracket c \rrbracket \circ \llbracket b \rrbracket.$$

The loop may execute its body any finite number of times, hence

$$\llbracket \text{while } b \text{ do } c \rrbracket S = \llbracket \neg b \rrbracket \left(\bigcup_{i \geq 0} T^i(S) \right).$$

Here $T^i(S)$ contains the states at the loop head after exactly i iterations; their union collects every state that can reach the loop head.

The same loop-head set is $\text{lfp } \Phi_S$ (cf. Lecture 3), hence

$$\llbracket \text{while } b \text{ do } c \rrbracket S \triangleq \llbracket \neg b \rrbracket (\text{lfp } \Phi_S), \quad \text{with} \quad \Phi_S(X) \triangleq S \cup T(X).$$

Collecting semantics

$$\llbracket \text{skip} \rrbracket S \triangleq S$$

$$\llbracket x := a \rrbracket S \triangleq \text{assign}(x, a, S)$$

$$\llbracket b \rrbracket S \triangleq \text{filter}(b, S)$$

$$\llbracket c_1 ; c_2 \rrbracket S \triangleq \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket S)$$

$$\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket S \triangleq \llbracket c_1 \rrbracket (\llbracket b \rrbracket S) \sqcup \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket S)$$

$$\Phi_S(X) \triangleq S \sqcup \llbracket c \rrbracket (\llbracket b \rrbracket X)$$

$$\llbracket \text{while } b \text{ do } c \rrbracket S \triangleq \llbracket \neg b \rrbracket (\text{lfp} \Phi_S)$$

Abstract interpreter

$$\llbracket \text{skip} \rrbracket^\# A \triangleq A$$

$$\llbracket x := a \rrbracket^\# A \triangleq \text{assign}^\#(x, a, A)$$

$$\llbracket b \rrbracket^\# A \triangleq \text{filter}^\#(b, A)$$

$$\llbracket c_1 ; c_2 \rrbracket^\# A \triangleq \llbracket c_2 \rrbracket^\# (\llbracket c_1 \rrbracket^\# A)$$

$$\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket^\# A \triangleq \llbracket c_1 \rrbracket^\# (\llbracket b \rrbracket^\# A) \sqcup \llbracket c_2 \rrbracket^\# (\llbracket \neg b \rrbracket^\# A)$$

$$\Phi_A^\#(X) \triangleq A \sqcup \llbracket c \rrbracket^\# (\llbracket b \rrbracket^\# X)$$

$$\llbracket \text{while } b \text{ do } c \rrbracket^\# A \triangleq \llbracket \neg b \rrbracket^\# (\text{lfp} \Phi_A^\#)$$

Each $\llbracket \cdot \rrbracket^\#$ is designed as a [sound, computable approximation](#) of $\llbracket \cdot \rrbracket$.

Example: the collecting semantics of the running loop

Same $c \equiv \text{while } x \leq 2 \text{ do } x := x+1$. Only x matters, so we identify the state $[x \mapsto n]$ with n : $\Sigma \cong \mathbb{Z}$, hence $\mathcal{P}(\Sigma) \cong \mathcal{P}(\mathbb{Z})$. Start $S = \{0\}$, with

$$T \triangleq \llbracket x := x+1 \rrbracket \circ \llbracket x \leq 2 \rrbracket.$$

Iterating T from S :

$$\begin{array}{lll} T^0(S) = \{0\}, & T^1(S) = \{1\}, & T^2(S) = \{2\}, \\ T^3(S) = \{3\}, & T^4(S) = \emptyset. & \end{array}$$

States at the loop head: $\bigcup_{i \geq 0} T^i(S) = \{0, 1, 2, 3\}$ (the loop invariant, computed!). After the exit test:

$$\llbracket c \rrbracket \{0\} = \llbracket x > 2 \rrbracket \left(\bigcup_{i \geq 0} T^i(S) \right) = \{3\}.$$

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head

`while  $x \leq 2$  do`

`$x := x+1$`

▷ after body

`done`

▷ exit

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

```
▷ head           {0}
while  $x \leq 2$  do
   $x := x+1$ 
  ▷ after body
done
▷ exit
```

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\}$

while $x \leq 2$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\} \subseteq \{0, 1, 2\}$

while $x \leq 2$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\} \subseteq \{0, 1, 2\} \subseteq \{0, 1, 2, 3\}$

while $x \leq 2$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\} \subseteq \{0, 1, 2\} \subseteq \{0, 1, 2, 3\} = \bigcup_{i \geq 0} T^i(S)$

while $x \leq 2$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\} \subseteq \{0, 1, 2\} \subseteq \{0, 1, 2, 3\} = \bigcup_{i \geq 0} T^i(S)$

while $x \leq 2$ do

$x := x+1$

▷ after body $\{1, 2, 3\} = x+1$ on $\llbracket x \leq 2 \rrbracket$ head

done

▷ exit $\{3\} = \llbracket x > 2 \rrbracket$ head

Unrolling it *on the program*

Same run, entry $S = \{0\}$. We annotate each **program point** with the set of states reaching it. The loop head collects the states obtained after any finite number of iterations.

▷ head $\{0\} \subseteq \{0, 1\} \subseteq \{0, 1, 2\} \subseteq \{0, 1, 2, 3\} = \bigcup_{i \geq 0} T^i(S)$

while $x \leq 2$ do

$x := x+1$

▷ after body $\{1, 2, 3\} = x+1$ on $\llbracket x \leq 2 \rrbracket$ head

done

▷ exit $\{3\} = \llbracket x > 2 \rrbracket$ head

The loop-head set is $\{0, 1, 2, 3\}$ — the **invariant** an analyzer attaches to that point.

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

▷ head

`while  $x \leq y$  do`

`$x := x+1$`

▷ after body

`done`

▷ exit

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

```
▷ head            $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in \{2\} \}$   
while  $x \leq y$  do  
     $x := x+1$   
    ▷ after body  
done  
▷ exit
```

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

▷ head $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in \{2, 1\} \}$

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

```
▷ head            $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in \{2, 1, 0\} \}$   
while  $x \leq y$  do  
     $x := x+1$   
    ▷ after body  
done  
▷ exit
```

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

```
▷ head            $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in \{2, 1, 0, -1\} \}$   
while  $x \leq y$  do  
     $x := x+1$   
    ▷ after body  
done  
▷ exit
```

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{ (a, a+2) \mid a \in \mathbb{Z} \}$ (y is never modified). We keep every set in the form $\{ (a, a+d) \mid a \in \mathbb{Z}, d \in D \}$; the guard $x \leq y$ selects $d \geq 0$.

▷ head $\{(a, a+d) \mid a \in \mathbb{Z}, d \in \{2, 1, 0, -1\}\}$ fixpoint!

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Unrolling it *on the program* — an infinite S

We identify a state σ with the pair $(\sigma(x), \sigma(y)) \in \mathbb{Z}^2$. Now take the **infinite** entry set $S = \{(a, a+2) \mid a \in \mathbb{Z}\}$ (y is never modified). We keep every set in the form $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$; the guard $x \leq y$ selects $d \geq 0$.

```
▷ head            $\{(a, a+d) \mid a \in \mathbb{Z}, d \in \{2, 1, 0, -1\}\}$   fixpoint!
while  $x \leq y$  do
  x := x+1
  ▷ after body     $\{(a, a+d) \mid a \in \mathbb{Z}, d \in \{1, 0, -1\}\}$ 
done
▷ exit            $\{(a, a+d) \mid a \in \mathbb{Z}, d = -1\} = \{(a, a-1) \mid a \in \mathbb{Z}\}$ , i.e.  $x = y+1$ 
```

Remark 1 (In abstract interpretation, a set *is* a property)

*We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.*

▷ head

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

▷ head $\{y=x+2\}$

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

▷ head $\{y=x+2\} \subseteq \{1 \leq y-x \leq 2\}$

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

```
▷ head            $\{y=x+2\} \subseteq \{1 \leq y-x \leq 2\} \subseteq \{0 \leq y-x \leq 2\}$   
  
while  $x \leq y$  do  
     $x := x+1$   
▷ after body  
  
done  
  
▷ exit
```

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

▷ head $\{y=x+2\} \subseteq \{1 \leq y-x \leq 2\} \subseteq \{0 \leq y-x \leq 2\} \subseteq \{-1 \leq y-x \leq 2\}$

while $x \leq y$ do

$x := x+1$

▷ after body

done

▷ exit

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

```
▷ head            $\{y=x+2\} \subseteq \{1 \leq y-x \leq 2\} \subseteq \{0 \leq y-x \leq 2\} \subseteq \{-1 \leq y-x \leq 2\}$   fixpoint!
while  $x \leq y$  do
  x := x+1
▷ after body
done
▷ exit
```

Remark 1 (In abstract interpretation, a set *is* a property)

We freely identify a set of states $S \subseteq \Sigma$ with the predicate it denotes, $\sigma \mapsto (\sigma \in S)$. So the set $\{(a, a+d) \mid a \in \mathbb{Z}, d \in D\}$ is just a way to write a *property* of (x, y) — e.g. S is $\{y = x+2\}$. This is why the filter notation $S \cap \{b\}$ is coherent: $\{b\}$ is the boolean b read as a set/property.

▷ head $\{y=x+2\} \subseteq \{1 \leq y-x \leq 2\} \subseteq \{0 \leq y-x \leq 2\} \subseteq \{-1 \leq y-x \leq 2\}$ fixpoint!

while $x \leq y$ do

$x := x+1$

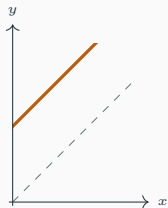
▷ after body $\{-1 \leq y-x \leq 1\}$

done

▷ exit $\{y-x = -1\}$ i.e. $x = y+1$

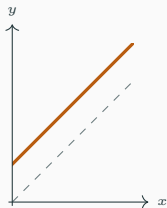
Geometric view of iterations

Recall that $T \triangleq \llbracket x := x + 1 \rrbracket \circ \llbracket x \leq y \rrbracket$.



$$T^0(S) = S$$

$$\{y = x + 2\}$$



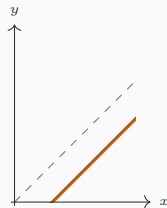
$$T^1(S) = T(S)$$

$$\{y = x + 1\}$$



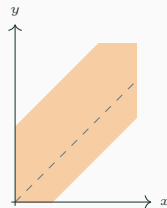
$$T^2(S)$$

$$\{y = x\}$$



$$T^3(S)$$

$$\{y = x - 1\}$$



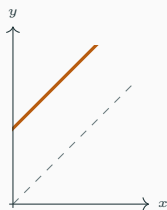
$$\bigcup_{i \geq 0} T^i(S)$$

$$\{-1 \leq y - x \leq 2\}$$

For every $k \geq 4$, $T^k(S) = \emptyset$.

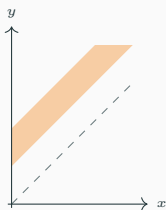
The same computation with Φ_S

Recall that $\Phi_S(X) \triangleq S \cup T(X)$: each iteration accumulates one more image of S by T .



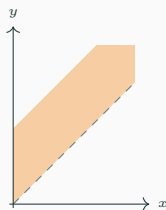
$$\Phi_S(\emptyset) = S$$

$$\{y = x + 2\}$$



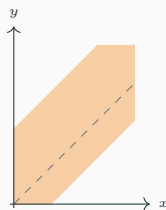
$$\Phi_S^2(\emptyset)$$

$$\{1 \leq y - x \leq 2\}$$



$$\Phi_S^3(\emptyset)$$

$$\{0 \leq y - x \leq 2\}$$



$$\Phi_S^4(\emptyset)$$

$$\{-1 \leq y - x \leq 2\}$$

fixpoint!

The sequence $(\Phi_S^k(\emptyset))_{k \geq 0}$ converges to the same fixpoint.

Exercise — another initial set

Same loop $c \equiv \text{while } x \leq y \text{ do } x := x+1$, entry $S = \{x \geq 1 \wedge x+2 \leq y\}$. Compute $\llbracket c \rrbracket S$ (loop-head set, then the exit) and draw the [geometric view](#).

Solution

The body lowers $y-x$ by 1 each step (while $x \leq y$). Over $\mathbb{Z}^2$ the shifted copies merge into one convex region: the head keeps $x \geq 1$, $y \geq 3$ and relaxes its lower bound on $y-x$.

▷ head $\{x \geq 1 \wedge y \geq 3 \wedge y-x \geq 2\}$

while $x \leq y$ do

$x := x+1$

done

▷ exit

Exercise — another initial set

Same loop $c \equiv \text{while } x \leq y \text{ do } x := x+1$, entry $S = \{x \geq 1 \wedge x+2 \leq y\}$. Compute $\llbracket c \rrbracket S$ (loop-head set, then the exit) and draw the [geometric view](#).

Solution

The body lowers $y-x$ by 1 each step (while $x \leq y$). Over $\mathbb{Z}^2$ the shifted copies merge into one convex region: the head keeps $x \geq 1$, $y \geq 3$ and relaxes its lower bound on $y-x$.

▷ head $\{x \geq 1 \wedge y \geq 3 \wedge y-x \geq 1\}$

while $x \leq y$ do

$x := x+1$

done

▷ exit

Exercise — another initial set

Same loop $c \equiv \text{while } x \leq y \text{ do } x := x+1$, entry $S = \{x \geq 1 \wedge x+2 \leq y\}$. Compute $\llbracket c \rrbracket S$ (loop-head set, then the exit) and draw the [geometric view](#).

Solution

The body lowers $y-x$ by 1 each step (while $x \leq y$). Over $\mathbb{Z}^2$ the shifted copies merge into one convex region: the head keeps $x \geq 1$, $y \geq 3$ and relaxes its lower bound on $y-x$.

▷ head $\{x \geq 1 \wedge y \geq 3 \wedge y-x \geq 0\}$

while $x \leq y$ do

$x := x+1$

done

▷ exit

Exercise — another initial set

Same loop $c \equiv \text{while } x \leq y \text{ do } x := x+1$, entry $S = \{x \geq 1 \wedge x+2 \leq y\}$. Compute $\llbracket c \rrbracket S$ (loop-head set, then the exit) and draw the [geometric view](#).

Solution

The body lowers $y-x$ by 1 each step (while $x \leq y$). Over $\mathbb{Z}^2$ the shifted copies merge into one convex region: the head keeps $x \geq 1$, $y \geq 3$ and relaxes its lower bound on $y-x$.

$$\triangleright \text{head} \quad \{x \geq 1 \wedge y \geq 3 \wedge y-x \geq -1\} = \text{lfp} \Phi_S$$

`while  $x \leq y$  do`

`$x := x+1$`

`done`

`$\triangleright$  exit`

Exercise — another initial set

Same loop $c \equiv \text{while } x \leq y \text{ do } x := x+1$, entry $S = \{x \geq 1 \wedge x+2 \leq y\}$. Compute $\llbracket c \rrbracket S$ (loop-head set, then the exit) and draw the [geometric view](#).

Solution

The body lowers $y-x$ by 1 each step (while $x \leq y$). Over $\mathbb{Z}^2$ the shifted copies merge into one convex region: the head keeps $x \geq 1$, $y \geq 3$ and relaxes its lower bound on $y-x$.

$$\triangleright \text{head} \quad \{x \geq 1 \wedge y \geq 3 \wedge y-x \geq -1\} = \text{lfp} \Phi_S$$

`while  $x \leq y$  do`

`$x := x+1$`

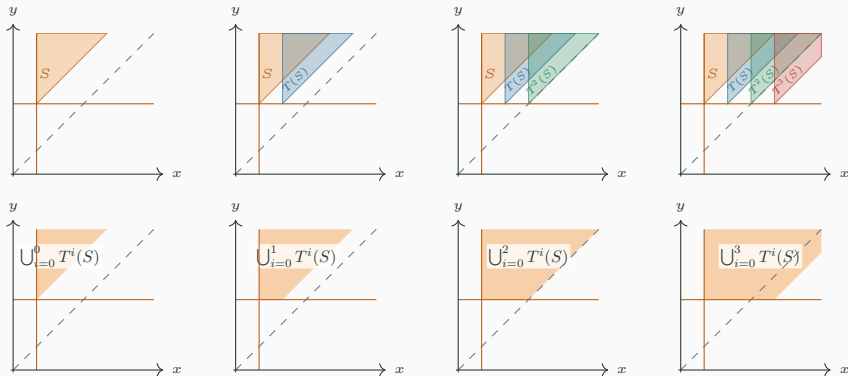
`done`

$$\triangleright \text{exit} \quad \{y \geq 3 \wedge y < x \leq y+1\}$$

The invariant is an (unbounded) [convex polyhedron](#) (3 constraints) — a preview of *polyhedra* (Lecture 7).

Exercise — geometric view of the iterates (solution)

The first row superposes the successive images of S ; the second shows their union. The apparent gaps between two images contain no point of $\mathbb{Z}^2$.



Summary

- Semantics = the mathematical meaning of a program; the reference against which *soundness* is defined.
- Three styles: **operational** (how), **denotational** (what function), **axiomatic** (what guarantee).
- **Rice**: all non-trivial semantic properties are undecidable $\Rightarrow$ approximation is mandatory.
- **Collecting semantics**: lift to sets of states, exact but uncomputable.

Next lecture: we will justify the various claims made here about fixpoints.

Standard semantics (operational / denotational / axiomatic)

- G. Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.
- H. R. Nielson, F. Nielson. *Semantics with Applications: An Appetizer*. Springer, 2007.

Collecting semantics

- It underlies essentially *all* of P. Cousot's papers on abstract interpretation.
- Gentle introductions:
 - A. Miné. *Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation*. Found. Trends Program. Lang. 4(3–4), 2017.
 - D. Cachera, D. Pichardie. *A Certified Denotational Abstract Interpreter*. ITP 2010.